

Bangladesh University of Business and Technology (BUBT)



Assignment

Course Code : CSE 324
Course Title : Compiler Design Lab
Assignment No. :

Name : Md. Fahim Islam
ID : 20245103035
Intake : 53
Section : 01
Program : B.Sc. in CSE

Name: Anusha Aziz

Dept. of
Bangladesh University of
Business and Technology

Date of Submission : 20/08/2026

Signature of Teacher

1. Valid Identifier

code:

```
%{
#include <stdio.h>
%}

%%
^[a-zA-Z_][a-zA-Z0-9_]*$    { printf("Valid Identifier\n"); }
.*                          { printf("Invalid Identifier\n"); }
%%

int main()
{
    yylex();
    return 0;
}
```

2. Valid Floating Point / Integer

code:

```
%{
#include <stdio.h>
%}

%%

^[0-9]+\.[0-9]+$    { printf("Valid Floating Point\n"); }
^[0-9]+$            { printf("Valid Integer\n"); }
.*                  { printf("Invalid Number\n"); }
%%

int main()
{
    yylex();
    return 0;
}
```

3. Identify All Tokens

code:

```
%{
#include <stdio.h>
%}

%%

"int"|"float"|"char"|"double"|"if"|"else"|"for"|"while"|"return"
    { printf("Keyword: %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]*
    { printf("Identifier: %s\n", yytext); }

[0-9]+(\.[0-9]+)?
    { printf("Number: %s\n", yytext); }

"=="|"!="|"<="|">="|"="|"+"|"-|"*"|"/"|"%"|"<"|">"
    { printf("Operator: %s\n", yytext); }

[O{};:,]
    { printf("Symbol: %s\n", yytext); }

[ \t\n]+ ;

"/" .* ;

.
    { printf("Unknown: %s\n", yytext); }

%%

int main()
{
    yylex();
    return 0;
}
```

4. String Matching: abb, a*b+, a.

code:

```
%{
#include <stdio.h>
%}

%%

^abb$    { printf("String belongs to abb\n"); }
^a*b+$   { printf("String belongs to a*b+\n"); }
^a.$     { printf("String belongs to a.\n"); }
.*       { printf("String does not match\n"); }

%%

int main()
{
    yylex();
    return 0;
}
```

5. Count Characters, Lines, Words, Spaces and Tabs

code:

```
%{
#include <stdio.h>

int characters = 0;
int lines = 0;
int words = 0;
int spaces = 0;
int tabs = 0;
%}

%%

\n      { characters++; lines++; }
" "     { characters++; spaces++; }
\t      { characters++; tabs++; }
```

```
[A-Za-z0-9]+ { characters += yyleng; words++; }
. { characters++; }
```

```
%%
```

```
int main()
{
    yylex();

    printf("Characters: %d\n", characters);
    printf("Lines: %d\n", lines);
    printf("Words: %d\n", words);
    printf("Spaces: %d\n", spaces);
    printf("Tabs: %d\n", tabs);

    return 0;
}
```

6. YACC Arithmetic Expression Evaluator

Task6.y

code:

```
%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *s);
}%

%token NUMBER

%left '+' '-'
%left '*' '/' '%'
%left UMINUS

%%

input:
    expr '\n'
    {
```

```

    printf("Valid Arithmetic Expression\n");
    printf("Evaluated Value: %d\n", $1);
}
;

```

expr:

```

    expr '+' expr    { $$ = $1 + $3; }
| expr '-' expr    { $$ = $1 - $3; }
| expr '*' expr    { $$ = $1 * $3; }
| expr '/' expr    { $$ = $1 / $3; }
| expr '%' expr    { $$ = $1 % $3; }
| '-' expr %prec UMINUS { $$ = -$2; }
| '(' expr ')'      { $$ = $2; }
| NUMBER            { $$ = $1; }
;

```

%%

```

void yyerror(const char *s)
{
    printf("Invalid Arithmetic Expression\n");
}

```

```

int main()
{
    return yyparse();
}

```

Task6.1

code:

```

%{
#include "q6.tab.h"
#include <stdlib.h>
%}

```

%%

```

[0-9]+    { yylval = atoi(yytext); return NUMBER; }
[ \t]+    ;
\n        { return '\n'; }
.         { return yytext[0]; }

```

%%

7(I). YACC Program: Binary to Decimal

Task7i.y

code:

%{

#include <stdio.h>

int yylex(void);

void yyerror(const char *s);

%}

%union

{

int val;

}

%token <val> DIGIT

%type <val> binary

%%

start:

binary '\n'

{

printf("Decimal Value: %d\n", \$1);

}

;

binary:

binary DIGIT { \$\$ = \$1 * 2 + \$2; }

| DIGIT { \$\$ = \$1; }

;

%%

void yyerror(const char *s)

{

```
    printf("Invalid Binary Value\n");
}
```

```
int main()
{
    return yyparse();
}
```

Task7i.l

code:

```
%{
#include "q7_binary.tab.h"
}%

%%

[01]    { yylval.val = yytext[0] - '0'; return DIGIT; }
[ \t]+  ;
\n      { return '\n'; }
.       { return yytext[0]; }

%%
```

7(II). YACC Program: Octal to Decimal

Task7ii.y

code:

```
%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *s);
}%

%union
{
    int val;
}

%token <val> DIGIT
```



```
%type <val> octal
```

```
%%
```

```
start:
```

```
    octal '\n'
    {
        printf("Decimal Value: %d\n", $1);
    }
    ;
```

```
octal:
```

```
    octal DIGIT { $$ = $1 * 8 + $2; }
    | DIGIT     { $$ = $1; }
    ;
```

```
%%
```

```
void yyerror(const char *s)
```

```
{
    printf("Invalid Octal Value\n");
}
```

```
int main()
```

```
{
    return yyparse();
}
```

Task7ii.1

code:

```
%{
#include "q7_octal.tab.h"
%}
```

```
%%
```

```
[0-7]    { yylval.val = yytext[0] - '0'; return DIGIT; }
[ \t]+   ;
\n       { return '\n'; }
.        { return yytext[0]; }
```

```
%%
```

7(III). YACC Program: Hexadecimal to Decimal

Task7iii.y

code:

```
%{
```

```
#include <stdio.h>
```

```
int yylex(void);
```

```
void yyerror(const char *s);
```

```
%}
```

```
%union
```

```
{
```

```
    int val;
```

```
}
```

```
%token <val> HEX
```

```
%type <val> hexadecimal
```

```
%%
```

```
start:
```

```
    hexadecimal '\n'
```

```
{
```

```
    printf("Decimal Value: %d\n", $1);
```

```
}
```

```
;
```

```
hexadecimal:
```

```
    hexadecimal HEX { $$ = $1 * 16 + $2; }
```

```
    | HEX           { $$ = $1; }
```

```
;
```

```
%%
```

```
void yyerror(const char *s)
```

```
{
```

```
    printf("Invalid Hexadecimal Value\n");
}
```

```
int main()
{
    return yyparse();
}
```

Task7iii.1

code:

```
%{
#include "q7_hex.tab.h"
}%

%%

[0-9] {
    yylval.val = yytext[0] - '0';
    return HEX;
}

[A-F] {
    yylval.val = yytext[0] - 'A' + 10;
    return HEX;
}

[a-f] {
    yylval.val = yytext[0] - 'a' + 10;
    return HEX;
}

[ \t]+ ;
\n    { return '\n'; }
.      { return yytext[0]; }

%%
```

8. YACC Program for CFG

Grammar:

$S \rightarrow aSb \mid ab$

Task8.y

code:

```
%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *s);
%}

%%

start:
    S '\n'
    {
        printf("String Accepted\n");
    }
    ;

S:
    'a' S 'b'
    | 'a' 'b'
    ;

%%

void yyerror(const char *s)
{
    printf("String Not Accepted\n");
}
```

```
int main()
{
    return yyparse();
}
```

Task8.l

code:

```
%{
#include "q8.tab.h"
%}
```

```
%%
```

```
a    { return 'a'; }
b    { return 'b'; }
[ \t]+ ;
\n   { return '\n'; }
.    { return yytext[0]; }
```

```
%%
```

9. YACC Program to Check Palindrome

Task9.y

code:

```
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
%}

%union
{
    char *str;
}

%token <str> WORD

%%

start:
    WORD '\n'
    {
        int i = 0;
        int j = strlen($1) - 1;
        int flag = 1;
```

```

while (i < j)
{
    if ($1[i] != $1[j])
    {
        flag = 0;
        break;
    }

    i++;
    j--;
}

if (flag)
    printf("Palindrome\n");
else
    printf("Not Palindrome\n");

free($1);
}
;

%%

```

```

void yyerror(const char *s)
{
    printf("Invalid Input\n");
}

```

```

int main()
{
    return yyparse();
}

```

Task9.1

code:

```

%{
#include "q9.tab.h"
#include <string.h>
%}

```

```
%%
```

```
[A-Za-z]+  
{  
    yylval.str = strdup(yytext);  
    return WORD;  
}
```

```
[ \t]+ ;  
\n    { return '\n'; }  
.    { return yytext[0]; }
```

```
%%
```

10(I). C Program for LL(1) Parsing

```
#include <stdio.h>  
#include <string.h>
```

```
char *table(char nonterminal, char terminal)  
{  
    if (nonterminal == 'E' &&  
        (terminal == 'i' || terminal == '('))  
        return "TR";  
  
    if (nonterminal == 'R' && terminal == '+')  
        return "+TR";  
  
    if (nonterminal == 'R' &&  
        (terminal == ')' || terminal == '$'))  
        return "";  
  
    if (nonterminal == 'T' &&  
        (terminal == 'i' || terminal == '('))  
        return "FU";  
  
    if (nonterminal == 'U' && terminal == '*')  
        return "*FU";  
  
    if (nonterminal == 'U' &&
```

```

        (terminal == '+' || terminal == ')' || terminal == '$'))
        return "";

    if (nonterminal == 'F' && terminal == 'i')
        return "i";

    if (nonterminal == 'F' && terminal == '(')
        return "(E)";

    return NULL;
}

int main()
{
    char input[100];
    char stack[100] = "$E";
    char *production;

    int top = 2;
    int i = 0;

    scanf("%99s", input);
    strcat(input, "$");

    while (top > 0)
    {
        char X = stack[--top];
        char a = input[i];

        if (X == '$')
        {
            if (a == '$')
                break;

            printf("Rejected by LL(1) Parser\n");
            return 0;
        }

        if (X == 'i' || X == '+' || X == '*' ||
            X == '(' || X == ')')

```



```

{
    if (X == a)
        i++;
    else
    {
        printf("Rejected by LL(1) Parser\n");
        return 0;
    }
}
else
{
    production = table(X, a);

    if (production == NULL)
    {
        printf("Rejected by LL(1) Parser\n");
        return 0;
    }

    for (int j = strlen(production) - 1; j >= 0; j--)
        stack[top++] = production[j];
}

printf("Accepted by LL(1) Parser\n");

return 0;
}

```

10(II). C Program for Recursive Descent Parsing

```
#include <stdio.h>
```

```
char *input;
int pos = 0;
```

```
void E();
void R();
void T();
```

```
void U();  
void F();
```

```
void E()  
{  
    T();  
    R();  
}
```

```
void R()  
{  
    if (input[pos] == '+')  
    {  
        pos++;  
        T();  
        R();  
    }  
}
```

```
void T()  
{  
    F();  
    U();  
}
```

```
void U()  
{  
    if (input[pos] == '*')  
    {  
        pos++;  
        F();  
        U();  
    }  
}
```

```
void F()  
{  
    if (input[pos] == 'i')  
    {  
        pos++;  
    }  
}
```

```

    }
    else if (input[pos] == '(')
    {
        pos++;
        E();

        if (input[pos] == ')')
            pos++;
        else
            pos = 9999;
    }
    else
    {
        pos = 9999;
    }
}

int main()
{
    char str[100];

    scanf("%99s", str);

    input = str;

    E();

    if (input[pos] == '\0' && pos < 9999)
        printf("Accepted by Recursive Descent Parser\n");
    else
        printf("Rejected by Recursive Descent Parser\n");

    return 0;
}

```

11. Convert BNF to YACC and Generate Abstract Syntax Tree

BNF

```

<stmt> ::= <id> = <expr> ;
<expr> ::= <expr> + <term> | <expr> - <term> | <term>
<term> ::= <term> * <factor> | <term> / <factor> | <factor>

```

<factor> ::= (<expr>) | <id> | <number>

Task11.y

code:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node
{
    char *value;
    struct Node *left;
    struct Node *right;
} Node;

Node *createNode(char *value, Node *left, Node *right)
{
    Node *node = malloc(sizeof(Node));

    node->value = strdup(value);
    node->left = left;
    node->right = right;

    return node;
}

void printTree(Node *root, int level)
{
    if (root == NULL)
        return;

    for (int i = 0; i < level; i++)
        printf(" ");

    printf("%s\n", root->value);

    printTree(root->left, level + 1);
    printTree(root->right, level + 1);
}
```

```
int yylex(void);
void yyerror(const char *s);
%}
```

```
%union
{
    char *str;
    Node *node;
}
```

```
%token <str> ID NUM
```

```
%type <node> program stmt expr term factor
```

```
%left '+' '-'
%left '*' '/'
```

```
%%
```

```
program:
    stmt
    {
        printf("Abstract Syntax Tree:\n");
        printTree($1, 0);
    }
    ;
```

```
stmt:
    ID '=' expr ';'
    {
        $$ = createNode("=", createNode($1, NULL, NULL), $3);
    }
    ;
```

```
expr:
    expr '+' term
    {
        $$ = createNode("+", $1, $3);
    }
    | expr '-' term
```

```

    {
        $$ = createNode("-", $1, $3);
    }
| term
    {
        $$ = $1;
    }
;

```

term:

```

    term '*' factor
    {
        $$ = createNode("*", $1, $3);
    }
| term '/' factor
    {
        $$ = createNode("/", $1, $3);
    }
| factor
    {
        $$ = $1;
    }
;

```

factor:

```

    '(' expr ')'
    {
        $$ = $2;
    }
| ID
    {
        $$ = createNode($1, NULL, NULL);
    }
| NUM
    {
        $$ = createNode($1, NULL, NULL);
    }
;

```

%%

```
void yyerror(const char *s)
{
    printf("Invalid Syntax\n");
}
```

```
int main()
{
    return yyparse();
}
```

Task11.1

code:

```
%{
#include "q11.tab.h"
#include <string.h>
%}
```

```
%%
```

```
[a-zA-Z_][a-zA-Z0-9_]*
{
    yylval.str = strdup(yytext);
    return ID;
}
```

```
[0-9]+
{
    yylval.str = strdup(yytext);
    return NUM;
}
```

```
[ \t\n]+ ;
```

```
. { return yytext[0]; }
```

```
%%
```

12. Generate Machine Code from AST

Task12.y

code:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node
{
    char *value;
    struct Node *left;
    struct Node *right;
} Node;

Node *createNode(char *value, Node *left, Node *right)
{
    Node *node = malloc(sizeof(Node));

    node->value = strdup(value);
    node->left = left;
    node->right = right;

    return node;
}

int reg = 0;

int generateCode(Node *root)
{
    if (root->left == NULL && root->right == NULL)
    {
        int r = reg++;

        printf("MOV R%d, %s\n", r, root->value);

        return r;
    }

    int leftReg = generateCode(root->left);
```



```

    int rightReg = generateCode(root->right);

    if (strcmp(root->value, "+") == 0)
        printf("ADD R%d, R%d\n", leftReg, rightReg);

    else if (strcmp(root->value, "-") == 0)
        printf("SUB R%d, R%d\n", leftReg, rightReg);

    else if (strcmp(root->value, "*") == 0)
        printf("MUL R%d, R%d\n", leftReg, rightReg);

    else if (strcmp(root->value, "/") == 0)
        printf("DIV R%d, R%d\n", leftReg, rightReg);

    return leftReg;
}

int yylex(void);
void yyerror(const char *s);
%}

%union
{
    char *str;
    Node *node;
}

%token <str> ID NUM

%type <node> stmt expr term factor

%left '+' '-'
%left '*' '/'

%%

stmt:
    ID '=' expr ';'
    {
        int r = generateCode($3);

```

```
    printf("STORE %s, R%d\n", $1, r);  
}  
;
```

expr:

```
    expr '+' term  
    {  
        $$ = createNode("+", $1, $3);  
    }  
| expr '-' term  
    {  
        $$ = createNode("-", $1, $3);  
    }  
| term  
    {  
        $$ = $1;  
    }  
;
```

term:

```
    term '*' factor  
    {  
        $$ = createNode("*", $1, $3);  
    }  
| term '/' factor  
    {  
        $$ = createNode("/", $1, $3);  
    }  
| factor  
    {  
        $$ = $1;  
    }  
;
```

factor:

```
    '(' expr ')'  
    {  
        $$ = $2;  
    }  
| ID
```

```

    {
        $$ = createNode($1, NULL, NULL);
    }
| NUM
    {
        $$ = createNode($1, NULL, NULL);
    }
;

```

%%

```

void yyerror(const char *s)
{
    printf("Invalid Syntax\n");
}

```

```

int main()
{
    return yyparse();
}

```

Task12.1

code:

```

%{
#include "q12.tab.h"
#include <string.h>
}%

```

%%

```

[a-zA-Z_][a-zA-Z0-9_]*
{
    yylval.str = strdup(yytext);
    return ID;
}

```

```

[0-9]+
{
    yylval.str = strdup(yytext);
    return NUM;
}

```

```
[ \t\n]+ ;
```

```
.      { return yytext[0]; }
```

```
%%
```